

Knowledge Graph-Based Clinical Inference System



Team CHEERS

지도교수 (Advisor): 송길태 (GILTAE SONG)

BYAMBASUREN TUVSHINJARGAL (202355673) tuvshinjargal@pusan.ac.kr
GALBADRAKH BUYANDELGER (202255630) buyandelger@pusan.ac.kr
BAVUUJAV DELGERBAYAR (202355622) bayar22@pusan.ac.kr

Abstract

This paper presents the design, methodology, and implementation plan for a knowledge graph-based clinical inference system that integrates biomedical entities — symptoms, diseases, and drugs — into a unified relational graph. The system performs multi-hop reasoning to infer probable diseases from user-provided symptoms, identify candidate treatment drugs, and issue medication safety warnings for drug–drug interactions (DDI). We employ knowledge graph embedding (KGE) models, specifically TransE and RotatE, trained on structured triples extracted from PrimeKG, DrugBank, and DDInter. The final deliverable is a web-based prototype comprising a FastAPI backend and a React frontend with interactive graph visualization. Evaluation uses standard link-prediction metrics including Hits@K and Mean Reciprocal Rank (MRR).

Keywords: knowledge graph, biomedical inference, knowledge graph embedding, TransE, RotatE, drug–drug interaction, symptom-disease mapping, clinical decision support, PyKEEN, FastAPI, React

Contents

1	Introduction	3
1.1	Research Motivation	3
1.2	System Objectives	3
2	Background and Related Work	3
2.1	Knowledge Graphs in Biomedicine	3
2.2	Knowledge Graph Embedding Models	3
2.3	Model comparison	3
3	System Architecture	4
4	Data Collection and Knowledge Graph Construction	4
4.1	Data Sources	5
4.2	Data Processing Pipeline	8
4.3	Knowledge Graph Construction	8
5	Model Implementation and Training	9
5.1	Knowledge Graph Embedding Theory	9
5.2	Training configuration	9
5.3	Inference Module	10
5.4	Evaluation metrics	11
6	Backend and Frontend Integration	11
6.1	Fast-API Backend	11
6.2	React Frontend core	12
6.3	Graph Visualization	13
7	Team Roles and Project Timeline	13
8	Conclusion	14

1. Introduction

Biomedical knowledge is inherently relational. Symptoms are associated with diseases, diseases are linked to treatments, and drugs can interact with one another in clinically significant ways. However, this information is typically distributed across heterogeneous databases and used in isolation, making integrated multi-step reasoning difficult for clinicians and patients alike. This project addresses that gap by representing biomedical relationships as a unified knowledge graph and applying knowledge graph embedding (KGE) models to perform inference across entity and relation types. The result is a system that can trace a reasoning chain from symptom input through disease prediction to drug recommendation, while flagging potential drug–drug interactions against a patient’s current medication.

1.1 Research Motivation

Clinical decision support tools increasingly rely on structured medical knowledge. Graph-based representations offer a natural fit for biomedical data because they explicitly encode the semantic relationships between entities. Unlike flat lookup tables, a knowledge graph can support multi-hop inference — for example, connecting a symptom to a disease to a treatment drug in a single reasoning chain. This explainability is a key advantage over black-box machine learning models.

1.2 System Objectives

- Infer probable diseases from one or more user-provided symptoms using trained KGE models.
- Identify candidate treatment drugs associated with the inferred diseases.
- Detect and warn about drug–drug interactions between candidate drugs and the patient’s current medication.
- Deliver a web-based prototype demonstrating the full reasoning pipeline.
- Evaluate model performance using standard link-prediction metrics (Hits@K, MRR).

2. Background and Related Work

2.1 Knowledge Graphs in Biomedicine

Biomedical knowledge graphs have emerged as a powerful paradigm for representing complex relationships between medical entities. PrimeKG (Chandak et al., 2023) integrates 20 biomedical databases into a multimodal graph with over 129,000 nodes and 8 million edges across 17 relation types. DrugBank provides curated drug–disease indication data for over 14,000 approved drugs. DDInter catalogs over 1.1 million drug–drug interaction pairs with severity classifications.

2.2 Knowledge Graph Embedding Models

KGE models learn low-dimensional vector representations of entities and relations such that valid triples score higher than invalid ones. TransE models relations as translations in embedding space: a triple (h, r, t) is considered valid when $h + r \approx t$. RotatE extends this idea by modeling relations as rotations in complex space, which better captures symmetric and antisymmetric relation patterns commonly observed in biological data.

2.3 Model comparison

Model	Score function	Relation patterns	Complexity	Recommended loss
TransE	$\ \mathbf{h} + \mathbf{r} - \mathbf{t}\ $	Hierarchical	$O(n \cdot d)$	Margin ranking
RotatE	$\ \mathbf{h} \circ \mathbf{r} - \mathbf{t}\ $	Sym, antisym, inverse	$O(n \cdot d)$	NSSA
ComplEx	$\text{Re}(\langle \mathbf{h}, \mathbf{r}, \mathbf{t} \rangle)$	Sym, antisym	$O(n \cdot d)$	Binary cross-entropy

Table 1: Comparison of knowledge graph embedding models evaluated in this project.

3. System Architecture

The system follows a three-tier architecture: a React-based frontend, a FastAPI backend, and a knowledge graph inference engine powered by a trained KGE model. The inference pipeline executes three sequential tasks (symptom-to-disease, disease-to-drug, and DDI check) producing structured JSON results that the frontend renders as a disease card list, drug recommendation list, warning banners, and an interactive Cytoscape.js graph visualization.

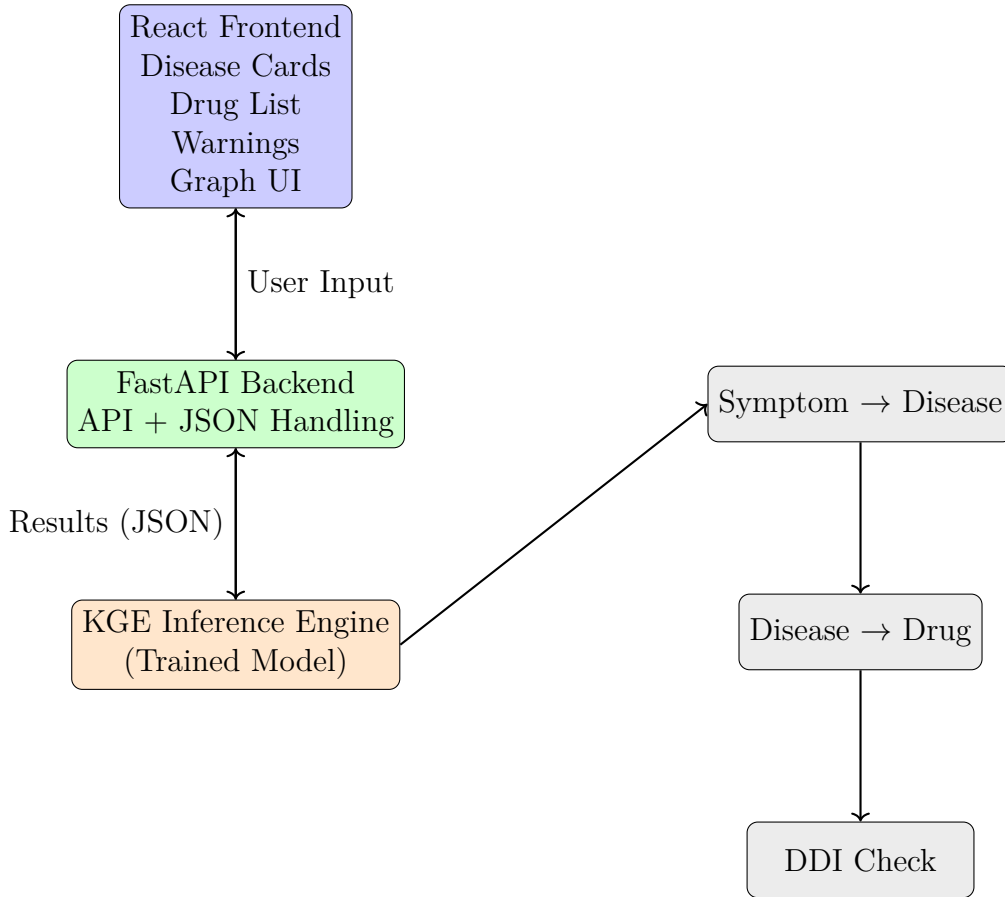


Figure 1: Three-tier architecture and inference pipeline of the system.

4. Data Collection and Knowledge Graph Construction

Phase 1 is the responsibility of Member 1 and constitutes the data engineering foundation of the entire project. Its output is a single unified triple file that Member 2 uses directly for model training. The quality and consistency of this file determines the ceiling of model performance.

4.1 Data Sources

All data will be collected from publicly available or academically accessible biomedical datasets.

- **PrimeKG (Harvard, open access):** Provides symptom–disease edges via the `disease_phenotype` relation. Contains approximately 129,000 nodes and 8 million edges across 17 relation types. Downloaded from github.com/mims-harvard/PrimeKG.

```
import pandas as pd

kg = pd.read_csv('kg.csv')

print(kg.shape)
(8100498, 12)
```

8,100,498 rows — each row represents one edge (relationship) in the knowledge graph. There are over 8 million edges in total.

12 columns — each row contains 12 pieces of information describing that edge. However, we only need approximately 5 of those columns.

```
print(kg.columns.tolist())

['relation', 'display_relation', 'x_index', 'x_id',
 'x_type', 'x_name', 'x_source', 'y_index', 'y_id',
 'y_type', 'y_name', 'y_source']
```

relation

The edge type. This is the most important column because it is used to filter and extract triples from the knowledge graph.

x_name

Human-readable name of the head entity, such as “Fever” or “Influenza”.

y_name

Human-readable name of the tail entity.

x_type

Category of the head entity, for example: “disease”, “drug”, “gene”, “protein”, or “phenotype”.

y_type

Category of the tail entity.

```
kg['relation'].value_counts()

disease_phenotype      142388
protein_protein        642150
drug_disease           45211
contraindication       38701
off_label_use          26551
phenotype_protein      38904
drug_protein           93401
```

disease_protein	160822
biological_process	107157
exposure_disease	11280
exposure_protein	61237
molfunc_protein	97222
cellcomp_protein	77213
anatomy_protein_present	127858
anatomy_protein_absent	73248
disease_disease	40397
indication	18776
...	

dtype: int64

disease__phenotype (142,388)

Used as the primary source for **Symptom** → **Disease** relationships.

protein__protein (642,150)

Represents interactions between proteins.

drug__disease (45,211)

Used as a source for **Disease** → **Drug** relationships.

contraindication (38,701)

Represents drugs or treatments that should not be used under certain conditions.

off_label__use (26,551)

Describes drugs used for purposes outside their officially approved indications.

phenotype__protein (38,904)

Represents relationships between phenotypes and proteins.

drug__protein (93,401)

Represents interactions between drugs and proteins.

disease__protein (160,822)

Represents associations between diseases and proteins.

biological__process (107,157)

Represents biological process relationships.

exposure__disease (11,280)

Represents disease relationships caused by environmental or external exposure.

exposure__protein (61,237)

Represents exposure-related protein interactions.

molfunc__protein (97,222)

Represents molecular function annotations for proteins.

cellcomp__protein (77,213)

Represents cellular component relationships involving proteins.

anatomy__protein__present (127,858)

Represents proteins present in specific anatomical structures.

anatomy__protein__absent (73,248)

Represents proteins absent in specific anatomical structures.

disease__disease (40,397)

Represents relationships between diseases.

indication (18,776)

Used for **Disease** → **Drug** treatment indication relationships.

`kg.head()`

Simplified preview of the output table using the most important columns:

#	relation	x_name	x_type	y_name
0	disease_phenotype	Influenza	disease	Fever
1	disease_phenotype	Influenza	disease	Cough
2	drug_disease	Oseltamivir	drug	Influenza

Table 2: Simplified Knowledge Graph Table

Explanation:

- **relation** describes the relationship type between entities.
- **x_name** represents the head entity.
- **x_type** represents the category of the head entity.
- **y_name** represents the tail entity.
- **y_type** represents the category of the tail entity.

Row 0 and Row 1

Influenza → **Fever** and **Influenza** → **Cough** represent disease-to-symptom relationships. These are exactly the triples used for **Symptom** → **Disease** prediction tasks.

In these rows:

- **x_name** represents the disease entity.
- **y_name** represents the phenotype or symptom entity.

Although the original direction in the dataset is:

Disease → **Symptom**

it can be reversed into:

Symptom → **Disease**

depending on the application or prediction objective.

Row 2

Oseltamivir → **Influenza** represents a **drug-to-disease** relationship.

This edge can be reversed to create a:

Disease → **Drug**

relationship using the semantic meaning **treated_by**.

For example:

Influenza $\xrightarrow{\text{treated_by}}$ Oseltamivir

- **DrugBank (academic registration required):** Provides disease–drug indication pairs (`treated_by` relation) for over 14,000 approved drugs. Requires a free academic account. Available as full database XML or open-access CSV files.
- **DDInter (open access):** Provides over 1.1 million drug–drug interaction pairs with severity classifications (Major, Moderate, Minor) and mechanism descriptions. Downloaded from `ddinter.scbdd.com` as lettered CSV files (A through Z).

4.2 Data Processing Pipeline

Raw data from all three sources undergoes a standardized cleaning pipeline to ensure entity name consistency across datasets. The central challenge is that the same biomedical entity may appear under different names, capitalizations, or synonyms across databases (e.g., 'Acetaminophen' vs 'Paracetamol'). The cleaning function below normalizes all names to lowercase underscore format.

Biomedical knowledge will be represented as triples in standard knowledge graph format:

(symptom, `related_to`, disease)
 (disease, `treated_by`, drug)
 (drug, `interacts_with`, drug)

4.3 Knowledge Graph Construction

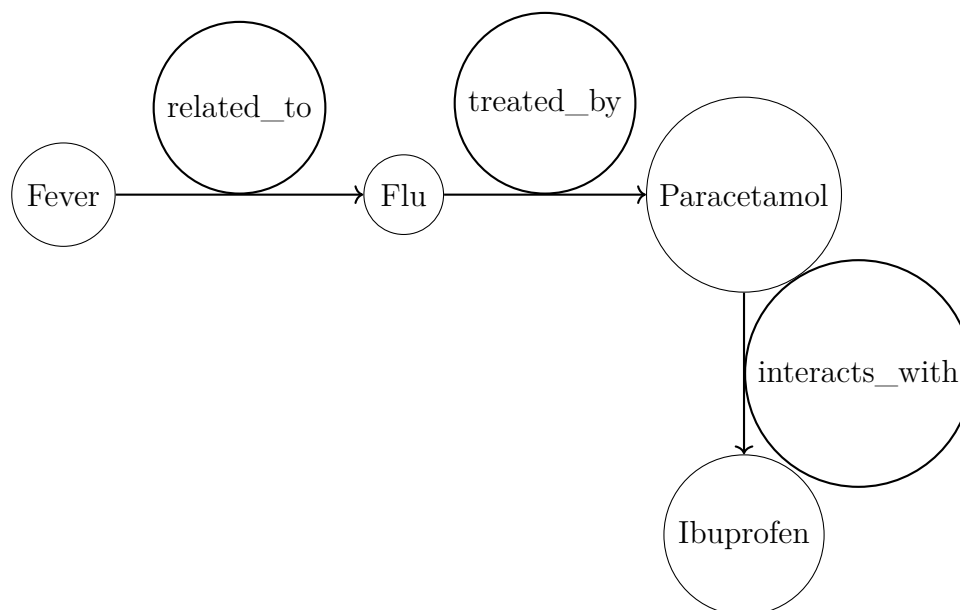


Figure 2: Reasoning path in knowledge graph

After preprocessing and normalization, all entities and relations will be combined into a biomedical knowledge graph. Each node will correspond to a biomedical concept such as a symptom, disease, or drug, and each edge will represent a semantic relation between two nodes. This graph will serve as the basis for inference. By learning the structure of the graph, the system will be able to predict plausible but previously unseen links.

5. Model Implementation and Training

Phase 2 constitutes the machine learning core of the project. A single KGE model is trained on the unified triple file from Phase 1. The trained model serves all three inference tasks — symptom-to-disease, disease-to-drug, and DDI check — via a unified embedding space.

5.1 Knowledge Graph Embedding Theory

KGE models represent each entity e and relation r as a dense vector (embedding) in R^d , where d is the embedding dimension (typically 128–256). For a triple (h, r, t) , the model defines a scoring function $f(h, r, t)$ such that valid triples receive high scores and invalid ones receive low scores.

TransE defines the scoring function as

$$f(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|,$$

interpreting each relation as a translation vector in embedding space. Training minimizes a margin-ranking loss that enforces valid triples to score higher than corrupted triples (e.g., replacing the tail with a random entity) by at least a margin γ .

RotatE models each relation as an element-wise rotation in complex space:

$$f(h, r, t) = -\|\mathbf{h} \circ \mathbf{r} - \mathbf{t}\|,$$

where $\mathbf{h}, \mathbf{r}, \mathbf{t} \in C^d$. This formulation captures symmetry, antisymmetry, inversion, and composition patterns, which commonly arise in biomedical graphs (e.g., `interacts_with` is symmetric).

5.2 Training configuration

```

1 from pykeen.pipeline import pipeline
2 from pykeen.triples import TriplesFactory
3
4 # Load triples with consistent entity/relation mappings
5 tf_train = TriplesFactory.from_path('train.tsv')
6 tf_valid = TriplesFactory.from_path('valid.tsv',
7     entity_to_id=tf_train.entity_to_id,
8     relation_to_id=tf_train.relation_to_id)
9 tf_test = TriplesFactory.from_path('test.tsv',
10     entity_to_id=tf_train.entity_to_id,
11     relation_to_id=tf_train.relation_to_id)
12
13 # Train TransE baseline
14 result = pipeline(
15     training=tf_train, validation=tf_valid, testing=tf_test,
16     model='TransE',
17     model_kwargs=dict(embedding_dim=128),
18     optimizer='Adam',
19     optimizer_kwargs=dict(lr=0.001),
20     training_kwargs=dict(num_epochs=200, batch_size=512),
21     loss='MarginRankingLoss',
22     loss_kwargs=dict(margin=1.0),
23     evaluator_kwargs=dict(filtered=True),
24     stopper='early',
25     stopper_kwargs=dict(frequency=10, patience=5,
26                         metric='hits_at_10'),
27     random_seed=42,
```

```

28 )
29
30 result.save_to_directory('models/transe/')
31 print(result.metric_results.to_df())

```

Listing 1: Training a TransE model using PyKEEN

5.3 Inference Module

After training, the three inference functions are implemented as a standalone Python module that Member 3 imports directly into the FastAPI backend. Each function loads the saved model once at startup and runs scoring queries at request time

```

1 import torch, json
2 from pykeen.pipeline import PipelineResult
3
4 result      = PipelineResult.from_directory('models/rotate/')
5 model       = result.model.eval()
6
7 entity2id = json.load(open('entity2id.json'))
8 id2entity = {v: k for k, v in entity2id.items()}
9 relation2id = json.load(open('relation2id.json'))
10
11 def predict_tails(head_name, relation_name, top_k=5):
12     h = entity2id[head_name]
13     r = relation2id[relation_name]
14     n = len(entity2id)
15
16     hs = torch.tensor([h]*n)
17     rs = torch.tensor([r]*n)
18     ts = torch.arange(n)
19
20     with torch.no_grad():
21         scores = model.score_hrt(torch.stack([hs, rs, ts], dim=1))
22
23     top = scores.squeeze().argsort(descending=True)[:top_k]
24     return [(id2entity[i.item()], scores[i].item()) for i in top]
25
26 def infer_diseases(symptoms, top_k=5):
27     agg = {}
28     for s in symptoms:
29         for name, score in predict_tails(s, 'related_to', 20):
30             agg[name] = agg.get(name, 0) + score
31     return sorted(agg.items(), key=lambda x: -x[1])[:top_k]
32
33 def infer_drugs(diseases, top_k=5):
34     agg = {}
35     for dis, _ in diseases:
36         for name, score in predict_tails(dis, 'treated_by', 10):
37             agg[name] = max(agg.get(name, 0), score)
38     return sorted(agg.items(), key=lambda x: -x[1])[:top_k]
39
40 def check_ddi(candidate_drugs, current_med):
41     warnings = []
42     for drug, _ in candidate_drugs:

```

```

43     results = predict_tails(drug, 'interacts_with', 50)
44     hit = [r for r in results if r[0] == current_med]
45     if hit and hit[0][1] > 0.5:
46         warnings.append({
47             'drug': drug,
48             'interacts_with': current_med,
49             'score': round(hit[0][1], 3)
50         })
51     return warnings

```

Listing 2: Inference pipeline for disease, drug, and DDI prediction using a trained RotatE model

5.4 Evaluation metrics

Model performance is evaluated using filtered link prediction metrics. Filtered evaluation removes known valid triples from the ranking denominator, preventing penalization for predicting other correct answers. PyKEEN computes all metrics automatically at the end of each training run.

Metric	Definition	TransE (expected)	RotatE (expected)
Hits@1	Correct answer ranked #1	0.25-0.35	0.30-0.42
Hits@5	Correct in top 5 results	0.48-0.58	0.55-0.65
Hits@10	Correct in top 10 results	0.55-0.65	0.60-0.72
MRR	Mean of 1/rank across all queries	0.30-0.40	0.35-0.48

Table 3: Evaluation metrics and expected performance ranges for the two models.

6. Backend and Frontend Integration

Phase 3 is the responsibility of Member 3 and delivers the user-facing system. It consists of a FastAPI backend that exposes the inference module as a REST API, and a React frontend that provides the input form, results display, and interactive knowledge graph visualization. Member 3 must build with mock data from week 3 onward and integrate Member 2’s real model in weeks 8–9.

6.1 Fast-API Backend

```

1  from fastapi import FastAPI, HTTPException
2  from fastapi.middleware.cors import CORSMiddleware
3  from pydantic import BaseModel
4  from typing import Optional
5  from inference import infer_diseases, infer_drugs, check_ddi
6
7  app = FastAPI(title='Clinical_KG_Inference_API')
8
9  app.add_middleware(
10     CORSMiddleware,
11     allow_origins=['http://localhost:5173'],
12     allow_methods=['*'],
13     allow_headers=['*']
14 )
15

```

```

16 class InferRequest(BaseModel):
17     symptoms: list[str]
18     current_medication: Optional[str] = None
19     top_k: int = 5
20
21 @app.post('/infer')
22 def infer(req: InferRequest):
23     clean = [s.lower().replace(' ', '_') for s in req.symptoms]
24
25     diseases = infer_diseases(clean, req.top_k)
26     drugs     = infer_drugs(diseases, req.top_k)
27
28     warnings = []
29     if req.current_medication:
30         med = req.current_medication.lower().replace(' ', '_')
31         warnings = check_ddi(drugs, med)
32
33     return {
34         'diseases': [{'name': d, 'score': round(s, 3)} for d, s in
35                     diseases],
36         'drugs':    [{'name': d, 'score': round(s, 3)} for d, s in
37                     drugs],
38         'ddi_warnings': warnings,
39     }

```

Listing 3: FastAPI backend endpoint for clinical KG inference

6.2 React Frontend core

```

1 import { useState } from 'react'
2 import axios from 'axios'
3
4 const API = 'http://localhost:8000'
5
6 export default function App() {
7     const [symptoms, setSymptoms] = useState([])
8     const [input, setInput] = useState('')
9     const [medication, setMedication] = useState('')
10    const [results, setResults] = useState(null)
11    const [loading, setLoading] = useState(false)
12
13    const addSymptom = (e) => {
14        if (e.key !== 'Enter' || !input.trim()) return
15        setSymptoms(prev => [...prev, input.trim().toLowerCase()])
16        setInput('')
17    }
18
19    const runInference = async () => {
20        setLoading(true)
21        const { data } = await axios.post(`${API}/infer`, {
22            symptoms,
23            current_medication: medication || null
24        })
25        setResults(data)

```

```

26     setLoading(false)
27   }
28
29   return (
30     <div className='max-w-2xl mx-auto p-6'>
31       <input
32         value={input}
33         onChange={e => setInput(e.target.value)}
34         onKeyDown={addSymptom}
35         placeholder='Type symptom, press Enter'
36       />
37
38       {symptoms.map(s => <span key={s}>{s}</span>)}
39
40       <button onClick={runInference}>Run inference</button>
41
42       {loading && <p>Computing...</p>}
43       {results && <ResultsPanel data={results} />}
44     </div>
45   )
46 }

```

Listing 4: React frontend for symptom input and inference request

6.3 Graph Visualization

The frontend renders the full reasoning chain — symptom nodes, disease nodes, and drug nodes — as an interactive directed graph using Cytoscape.js. Nodes are color-coded by entity type. Drug nodes with DDI warnings receive a red border; safe drugs receive a green border. The layout uses breadth-first directed arrangement: symptoms on the left, diseases in the center, drugs on the right. Users can click any node to inspect its connections.

7. Team Roles and Project Timeline

Member	Name	ID	Primary responsibilities
Member 1	Byambasuren Tuvshinjargal	202355673	Data collection, cleaning, KG construction, entity normalization, triple validation, hand-off to Member 2
Member 2	Galbadrakh Buyandelger	202255630	Model implementation (TransE, RotatE), training, evaluation, inference module development, metric reporting
Member 3	Bavuujav Delgerbayar	202355622	FastAPI backend, React frontend, Cytoscape.js graph view, integration testing, demo preparation

Table 4: Team role assignment.

Member 1 is responsible for gathering and curating raw biomedical data from publicly accessible sources, normalizing entity names, and constructing the unified knowledge graph that underpins the entire system.

Member 2 leads the machine learning component of the project. This includes selecting, implementing, and training knowledge graph embedding models, running evaluations with standard metrics, and comparing alternative modeling strategies.

Member 3 handles the engineering side: designing the user-facing web interface, building the backend inference pipeline, connecting all components, and presenting prediction results in a clear and interactive format.

Tasks		5		6				7				8				9			
		3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Planning	Knowledge Graph Schema Definition																		
	Development Environment Setup																		
Data Engineering	Data Sourcing																		
	Data Cleaning																		
	Entity Normalization																		
	Triple Generation																		
Knowledge Graph Construction	Graph Database Implementation																		
	Triple Validation																		
	Query Optimization																		
AI/ML Modeling	Training Pipeline																		
	Hyperparameter Tuning																		
	Reasoning Logic																		
System Integration	Backend API Development																		
	Frontend Dashboard																		
	Graph Visualization																		
	Real-time Logic Connection																		
Validation and Finalization	Metric Evaluation																		
	Safety Recall Testing																		
	UX Refinement																		
	Documentation and Submission																		

Project Timeline

8. Conclusion

This project applies knowledge graph inference to a practical biomedical problem. By integrating symptom, disease, and drug entities from three curated databases into a unified graph, and training KGE models over this graph, the system enables multi-hop clinical reasoning: from raw symptoms through disease prediction to treatment recommendation and interaction safety checking. The methodological contribution lies in treating biomedical inference as a link-prediction problem over a heterogeneous multi-relational graph, rather than as a classification task over disconnected feature vectors. The practical contribution is a working web prototype that makes this reasoning transparent and accessible to non-expert users.

All three project phases — data engineering, model training, and system integration — are designed to be executed in parallel with progressive handoffs, enabling a 12-week delivery schedule for a team

References

- [1] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” *Advances in Neural Information Processing Systems*, vol. 26, 2013.
- [2] Z. Sun, Z. H. Deng, J. Y. Nie, and J. Tang, “RotatE: Knowledge graph embedding by relational rotation in complex space,” *International Conference on Learning Representations (ICLR)*, 2019.
- [3] P. Chandak, K. Huang, and M. Zitnik, “Building a knowledge graph to enable precision medicine,” *Scientific Data*, vol. 10, no. 1, p. 67, 2023.
- [4] M. Ali et al., “PyKEEN 1.0: A Python library for training and evaluating knowledge graph embeddings,” *Journal of Machine Learning Research*, vol. 22, no. 82, pp. 1–6, 2021.
- [5] D. S. Wishart et al., “DrugBank 5.0: A major update to the DrugBank database for 2018,” *Nucleic Acids Research*, vol. 46, no. D1, pp. D1074–D1082, 2018.
- [6] Y. H. Feng et al., “DDInter: An online drug–drug interaction database towards improving clinical decision-making and patient safety,” *Nucleic Acids Research*, vol. 50, no. D1, pp. D1200–D1207, 2022.
- [7] FastAPI, “FastAPI framework documentation,” 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [8] React, “React — A JavaScript library for building user interfaces,” 2024. [Online]. Available: <https://react.dev>
- [9] Cytoscape.js, “Cytoscape.js — Graph theory / network library for analysis and visualization,” 2024. [Online]. Available: <https://js.cytoscape.org>